Dispatching Requests to Controllers

At its most basic, a web application accepts an incoming request from a browser, processes it, and sends a response.

A question immediately springs to mind: how does the application know what to do with the incoming request? A shopping cart application will receive requests to display a catalog, add items to a cart, create an order, and so on. How does it route these requests to the appropriate code?

It turns out that Rails provides two ways to define how to route a request: a comprehensive way that you'll use when you need to and a convenient way that you'll generally use whenever you can.

The comprehensive way lets you define a direct mapping of URLs to actions based on pattern matching, requirements, and conditions. The convenient way lets you define routes based on resources, such as the models that you define. And because the convenient way is built on the comprehensive way, you can freely mix and match the two approaches.

In both cases, Rails encodes information in the request URL and uses a subsystem called Action Dispatch to determine what should be done with that request. The actual process is flexible, but at the end of it Rails has determined the name of the *controller* that handles this particular request along with a list of any other request parameters. In the process, either one of these additional parameters or the HTTP method itself is used to identify the *action* to be invoked in the target controller.

Rails routes support the mapping between URLs and actions based on the contents of the URL and on the HTTP method used to invoke the request. We've seen how to do this on a URL-by-URL basis using anonymous or named routes. Rails also supports a higher-level way of creating groups of related routes. To understand the motivation for this, we need to take a little diversion into the world of representational state transfer (REST).